TP N°4

Base de données avec SQL Workbench et Django

Objectifs du TP

À la fin de ce TP, vous serez capable de :

- Comprendre la relation entre **Models**, **ORM**, **BD**, et **Migrations**
- Créer un projet Django
- Configurer MySQL + MySQL Workbench avec Django
- Créer un modèle
- Générer et appliquer des migrations
- Visualiser les tables dans MySQL Workbench
- Faire des requêtes simples avec le ORM

Étape 1 : Préparer MySQL Workbench

1. Ouvrir SQL Workbench
   - Lancer SQL Workbench et se connecter à votre serveur MySQL.
   - Créer une nouvelle connexion si nécessaire (hostname: localhost, user: root, **password: votre_mot_de_passe**).
2. Créer une base de données :

   Dans MySQL Workbench, crée la BD : Bd_mysql

1re méthode :
Puis exécutez cette commande SQL dans MySQL 8.0 Command Line Client :

```
mysql> CREATE DATABASE tp_django;
Query OK, 1 row affected (0.01 sec)
```

ou

2e méthode : accédez à l'interface de MySQL Workbench, puis créez votre requête.

Cette commande permet d'afficher l'ensemble des tables présentes dans la base de données. Elle sert également à vérifier qu'une table a bien été créée.

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| exemple_bd         |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| tp_django          |
+--------------------+
6 rows in set (0.03 sec)
```

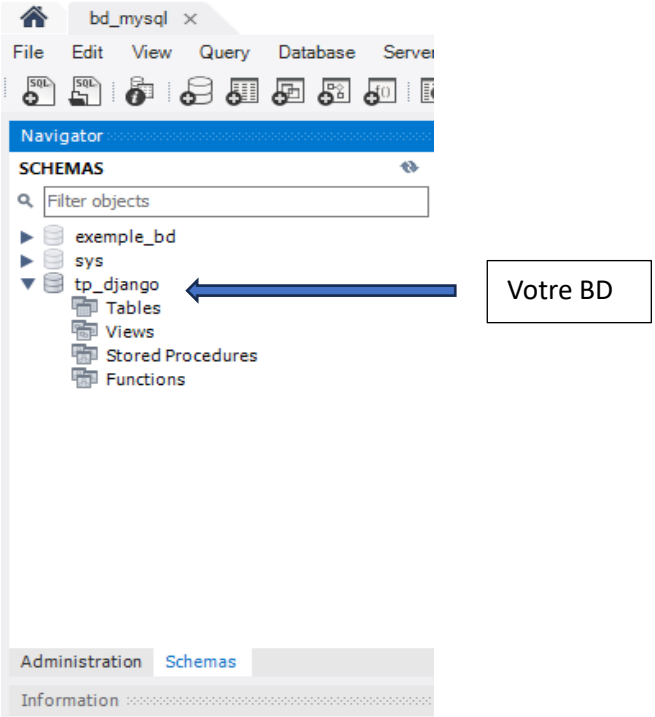
- Cette commande sélectionne la base tp_django.

USE est un mot-clé SQL qui sert à **sélectionner la base de données** sur laquelle tu vas travailler.

tp_django est le nom de la base de données que tu veux utiliser.

```
mysql> use tp_django;
Database changed
mysql> show tables;
Empty set (0.20 sec)
```

L'interface ci-dessous affiche la base de données **tp_django**, dans laquelle les tables ont été créées mais sont encore vides pour le moment.



Étape 2 : Configurer les paramètres MySQL de Django

Après avoir installé Django et configuré MySQL, vous devez configurer votre projet Django pour qu'il utilise la base de données MySQL. Ouvrez le fichier settings.py de votre projet Django, qui se trouve généralement dans le dossier du projet. Recherchez le paramètre DATABASES et mettez-le à jour avec la configuration MySQL :

- **Installer le connecteur MySQL**

  pip install mysqlclient

  ou si erreur :

  pip install pymysql

Étape 3 : Configurer settings.py pour MySQL

Ouvre tp_django/settings.py et remplace la section DATABASES par (adapter NAME, USER, PASSWORD selon ta config) :

settings.py × __init__.py ×

tp1 > settings.py > ...

```
82
83  DATABASES = {
84      'default': {
85          'ENGINE': 'django.db.backends.mysql',
86          'NAME': 'tp_django',
87          'USER': 'root',
88          'PASSWORD': 'root',
89          'HOST': '127.0.0.1',
90          'PORT': '3306',
91          'OPTIONS': {
92              'charset': 'utf8mb4',
93          }
94      }
95  }
```

Modifiez selon votre mot de passe.

```python
import pymysql
pymysql.install_as_MySQLdb()
```

⚠️ **Important : L'ORM Django va maintenant communiquer directement avec votre base MySQL.**

Étape 4 : Définir des modèles (models.py)

a. **Définissez un modèle dans ` FirstApp/models.py` :**

```python
from django.db import models
from django.utils import timezone

class Article(models.Model):
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_pub =  models.DateTimeField(default=timezone.now)
    def __str__(self):
        return self.titre
```

from django.db import models

- Ici, on importe le module models de Django.
- models contient toutes les classes et types de champs qu'on peut utiliser pour définir nos modèles (tables).

❖ **<u>Définition de la classe Article:</u>**

class Article(models.Model):

- Article est le **nom du modèle**, qui va correspondre à une **table** dans la base de données (Django créera une table appname_article automatiquement).
- models.Model signifie que Article hérite des fonctionnalités de Django pour un modèle (ORM). Cela permet de créer, lire, mettre à jour et supprimer des articles facilement.

- **<u>Champs du modèle:</u>**

titre = models.CharField(max_length=100)

- CharField : champ texte court.
- max_length=100 : limite le texte à 100 caractères.
- Sert à stocker le **titre de l'article**.

contenu = models.TextField()

- TextField : champ texte long, sans limite fixe.
- Sert à stocker le **contenu complet de l'article**.

date_pub = models.DateTimeField(auto_now_add=True)

- DateTimeField : champ date + heure.
  - auto_now_add=True : la date est automatiquement remplie avec la date et l'heure au moment de la création de l'article.
  - Sert à savoir quand l'article a été publié.

- **date_pub = models.DateTimeField(default=timezone.now)**

sert à enregistrer automatiquement la date et l'heure au moment où un article est créé.

- **DateTimeField**

C'est un **type de champ** dans Django qui permet d'enregistrer :

- la **date** (jour, mois, année)
- l'**heure** (heures, minutes, secondes)

Ce champ est utilisé lorsque tu veux savoir **quand** quelque chose a été créé, modifié ou publié.

C'est un **champ de la base de données** pour stocker une date + une heure (exemple : 2025-11-24 10:32:00).

- **default=timezone.now**
  Si tu ne donnes pas de date pour date_pub, Django va automatiquement mettre la date et l'heure actuelles.

- **<u>Méthode spéciale __str__</u>**

```
def __str__(self):
    return self.titre
```

- __str__ est une **méthode spéciale en Python** qui définit la représentation de l'objet quand on l'affiche.
- Ici, si vous faites print(Article), Django affichera le **titre de l'article** plutôt qu'un objet générique.

- Utile surtout dans l'**admin Django**, où vous verrez les titres au lieu de "Article object (1)".

Étape 4 — Générer et appliquer les migrations

1. **Créer les migrations (génère des fichiers dans FirstApp/migrations/) :**

Une fois on configure notre base de données dans le dictionnaire DATABASES dans le fichier setting.py, lancer les commandes de migrations :

La commande makemigrations permet de créer les scripts à exécuter dans votre base en fonctions de vos méthodes Model.

```
python manage.py makemigrations
```

La commande migrate exécute en base les définitions de vos modèles et de vos applications installées

```
python manage.py migrate
```

```
(.venv) PS C:\Users\asus\tp1_django> python manage.py migrate
Operations to perform:
  Apply all migrations: FirstApp, admin, auth, contenttypes, sessions
Running migrations:
  Applying FirstApp.0001_initial... OK
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
```

2.Vérifier l'état des migrations :

```
(.venv) PS C:\Users\asus\tp1_django> python manage.py showmigrations
FirstApp
 [X] 0001_initial
admin
 [X] 0001_initial
 [X] 0002_logentry_remove_auto_add
 [X] 0003_logentry_add_action_flag_choices
auth
 [X] 0001_initial
 [X] 0002_alter_permission_name_max_length
 [X] 0003_alter_user_email_max_length
 [X] 0004_alter_user_username_opts
 [X] 0005_alter_user_last_login_null
 [X] 0006_require_contenttypes_0002
 [X] 0007_alter_validators_add_error_messages
 [X] 0008_alter_user_username_max_length
 [X] 0009_alter_user_last_name_max_length
 [X] 0010_alter_group_name_max_length
 [X] 0011_update_proxy_permissions
 [X] 0012_alter_user_first_name_max_length
contenttypes
 [X] 0001_initial
 [X] 0002_remove_content_type_name
sessions
 [X] 0001_initial
```

python manage.py showmigrations

Cette commande sert à afficher toutes les migrations de ton projet Django.

- **Voir quelles migrations sont appliquées ou non appliquées**

Avec un symbole **[X]** ou **[]**

Exemple :

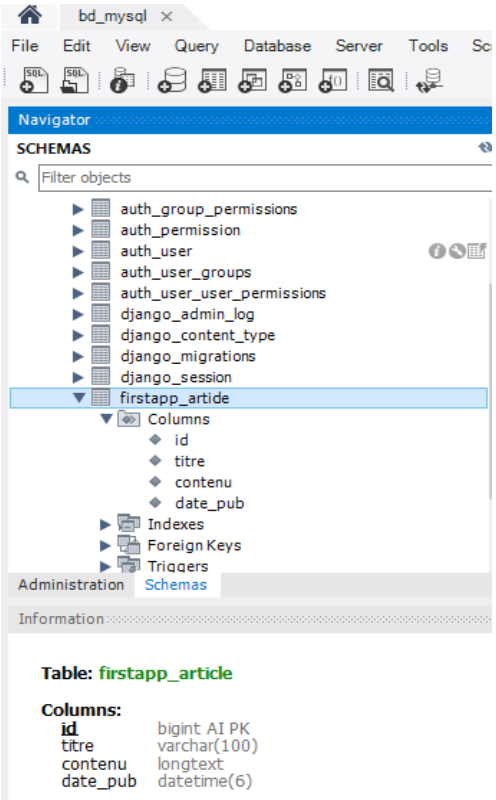
```
articles
 [X] 0001_initial
 [X] 0002_add_date
 [ ] 0003_add_author
```

- [X] = migration déjà exécutée dans la base de données
- [ ] = migration créée mais pas encore appliquée

❖ **Voir le SQL d'une migration donnée :**

```
(.venv) PS C:\Users\asus\tp1_django> python manage.py sqlmigrate FirstApp 0001
--
-- Create model Article
--
CREATE TABLE `FirstApp_article` (`id` bigint AUTO_INCREMENT NOT NULL PRIMARY KEY, `titre` varchar(100) NOT NULL, `contenu` longtext NOT NULL
, `date_pub` datetime(6) NOT NULL);
```

La figure ci-dessous présente la migration des tables dans la base de données tp_django.



La commande ci-dessous permet de vérifier toutes les tables de la base de données tp_django.

```
mysql> show tables;
+----------------------------+
| Tables_in_tp_django        |
+----------------------------+
| auth_group                 |
| auth_group_permissions     |
| auth_permission            |
| auth_user                  |
| auth_user_groups           |
| auth_user_user_permissions |
| django_admin_log           |
| django_content_type        |
| django_migrations          |
| django_session             |
| firstapp_article           |
+----------------------------+
11 rows in set (0.12 sec)
```